

Generative AI: retrieval and evaluation

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build `nw/policy`, Project 3: a policy service that answers from the current corpus, cites the passage, refuses when it should, and proves it with a harness whose baseline says what it was measured with. On the platform its prompt is a versioned record with an approval, its chunks live in your tenant's vector store, and every model call goes through the model gateway on your key.

Northwind can classify a ticket; nothing yet answers the customer

SERVICE LAYER

Foundations and
the AI service layer

PROJECT 1

Machine learning in
production

PROJECT 2

Deep learning and
representation

PROJECT 3

Generative AI:
retrieval and
evaluation

PROJECT 4

Agentic AI systems

CAPSTONE

Deployment,
observability and
capstone

Small models chose urgency and category for almost nothing, trained by a pipeline and approved in a registry. A question about the SLA needs a generator held to a source: an LLM with retrieval, citations and a refusal around it. The agent in the next part calls it as `search_policies` .

Today you build grounded answers with citations or a refusal, judged and gated

- 1 Read the corpus and the contract
- 2 Tokenise sensitive fields before indexing
- 3 Split on headings, carry the metadata
- 4 Hybrid retrieval, filters, a reranker
- 5 Citations enforced in code, or refuse
- 6 Prompts in the platform's prompt store
- 7 The harness, and the judge calibrated
- 8 The gate on every pull request, one experiment
- 9 The index in the platform's vector store
- 10 The service through the gateway: cache, feedback
- 11 The service knows when the questions change
- 12 The part's check and commit
- 13 Argue with the judge

28 documents, three superseded and four internal. 111 golden cases: 66 held out for the gate, 45 for tuning, 30 that must refuse, German, multi-hop and currency slices.

An LLM predicts tokens, so facts and currency are your job

What it is

- A model that predicts the next token (a word, a subword or a character), then the next, from everything before it
- General: one model drafts, summarises, extracts and classifies
- Steered by the prompt and by the context you put in front of it

What it is not

- A database: it keeps no record of which document said what
- Current: it knows its training data, not this quarter's SLA
- A source you can cite: a fluent answer and a true one look the same

Models hallucinate because training rewards a confident guess, so design for refusal

Training and benchmarks score a guess above "I do not know", so models learn to guess. The researchers propose scoring abstention instead.

Kalai et al., OpenAI and Georgia Tech, 2025

Even with the source in hand

1.8% to 11%

hallucination rate on grounded summarisation: the best model, then several frontier models (Vectara leaderboard, September 2026)

Today a refusal is a named outcome with a reason, and an answer with no valid citation is refused. That is the engineering answer to a guess.

Sensitive values never reach the index, the store, the prompt or the log

```
"Contact ops@bluefreight.example  
about account NW-11498"
```

```
"Contact [EMAIL_1]  
about account [ACCOUNT_1]"
```

Same value, same token within one text. Span-based: emails, phones with a country code, account and invoice ids, IPv4 and IPv6, keys, German patterns.

make pii-eval on 1,678 labelled spans	Recall	Precision
DETECTOR=none , patterns only	0.67	1.00
DETECTOR=heuristic , plus a name detector	0.98	1.00

The name recall is optimistic: the detector's first-name list overlaps the generator's. Measure on your own data before you trust it.

Chunking is an engineering decision, and the index remembers it

Strategy	Keeps	Loses	Use when
Fixed windows	nothing to configure	headings, section boundaries, metadata	unstructured prose
Heading - aware	section, title, metadata per chunk	nothing important on structured docs	policies, runbooks, contracts
Sentence windows with overlap	local context	document structure	long narrative text

A chunk is one heading's text carrying document, section, effective date, audience and `superseded_by`. The build writes `manifest.json`: corpus hash, chunker parameters, both models, 212 chunks, the prompt versions and a drift baseline. `build_index --check` exits 1 when the corpus has moved on.

Access is decided in the store, before anything is ranked

- 1 The caller's key sets the audience. The body can only narrow it; asking for `internal` without an internal key is a 403
- 2 Dense and BM25 candidates, scored only over chunks the audience may see. An unknown audience counts as internal: the filter fails closed
- 3 Fused with `rrf`: each list adds $1 / (60 + \text{rank})$, no weight to tune; superseded chunks dropped when `current_only`
- 4 A cross-encoder reranks the short list; the top `k` go to the model

Why before ranking

Filtered after ranking, an internal chunk still takes a candidate slot, and a filter bug returns it. Filtered in the store, it can do neither.

The reranker reads query and chunk together: most of the quality, most of the latency.

Citations are enforced in code, not requested in the prompt

- 1 Tell the model which chunk ids it may cite
- 2 Check every citation against the ids that were actually in the context
- 3 Drop invented ids and count them
- 4 No valid citation left: an answer from memory. Refuse it

Deterministic, so exactly 1.0

Citation validity in the harness must be 1.000. If it is not, the code is wrong, not the model.

Refusal is an outcome, not an error

Path	Reason on the response	Model called
Retrieval too weak to bother the model	<code>weak_retrieval</code>	no
The model says the context does not cover it	<code>model_unanswerable</code>	yes
No valid citation survived the check	<code>no_valid_citation</code>	yes
The track's guardrail blocked the question	<code>screened</code>	no

The duty manager rule for P0 tickets exists only in the internal escalation matrix. A customer asking gets a refusal. Only a caller whose key maps to `internal` gets the answer citing `escalation-matrix`.

A prompt is code the model runs, so it gets a version and an approval

```
policy.answer@6c2999977051
```

The name and the first twelve hex characters of the SHA-256 of the text. One changed word, a new hash.

```
ANSWER_PROMPT = register("policy.answer",  
SYSTEM)
```

- The hash rides on every answer, refusals included, every harness row, `eval.json`, the index manifest and the model-call span
- The platform's prompt store keeps the same hash as a version; `register` is idempotent
- `set_stage(..., APPROVED)` is the product owner's decision; `Stage.LIVE` belongs to the capstone
- Git versions the file; the hash versions what the running process sent

Evaluate retrieval and generation separately

Metric	Measures	Computed by
Recall@1, @3, @k, MRR	did the gold chunks make the list, and how high	deterministic
Key - fact correctness	the numbers, durations and names the gold answer carries, and no stale fact from a superseded policy	deterministic
Citation validity, citation recall	were the citations real, and did they cover the gold chunks	deterministic; validity must be 1.0
Refusal correctness	refused when it should, answered when it should	deterministic
Faithfulness	is every claim supported by the cited passages	the Judge, against a rubric

A bad answer from good retrieval is a prompt or model problem. A bad answer from bad retrieval is a chunking or index problem. One combined score hides which one you have.

Every rate comes with its count and an interval, because 66 cases is a small exam

Retrieval only, gate split	Measured
Recall@k	0.955, 63 of 66
Recall@1	0.879
MRR	0.923
Refusal correct	44 of 66, 95% CI 0.55 to 0.77
German slice, recall@k	0.700 on 10 cases

- **The split:** tune k, thresholds and the prompt on the 45 dev cases; the gate reads the 66 held-out cases. A threshold tuned on the gate split is tuned on the exam
- **Wilson interval:** a 95 percent range for a rate that stays honest near 0 and 1 and on small counts. 44 of 66 could be anything from 55 to 77 percent
- **Tolerances in cases:** the gate allows one case lost, not "0.03 of a rate" smaller than one case

A judge is an instrument, so calibrate it before you read it

- **102 labelled answers:** 30 by hand, 72 from real passages with a known perturbation: a changed number, the wrong plan, an outside fact
- **The report:** agreement, Cohen's kappa, false pass and false fail rates with counts and intervals, and the Judge's mean bias
- **False passes** bound what the gate can see; so does a bias interval wider than its tolerance
- `--from-eval` turns your Workhorse's answers into cases a person labels

Why an interval matters

The earlier 30-case set measured kappa 0.80 and 0 false passes out of 14. Zero out of 14 is compatible with a true false pass rate up to about 23 percent.

Opus grades gpt-oss, never the other way round.

The gate checks what a baseline was measured with before it compares a single number

Run	Calls	Baseline file
<code>--retrieval-only</code>	none	<code>policy-retrieval_only.json</code> , one for every track
<code>--no-judge</code>	Workhorse	<code>policy-<track>-no_judge.json</code>
<code>judged</code>	Workhorse, Judge	<code>policy-<track>-judged.json</code>

Provenance first: another mode, track, model id, prompt version, corpus, case set or Judge prompt fails with the reason. A deliberate model migration passes `--allow-model-change`, and the decision records it.

`eval-gate.yml`

retrieval on every pull request that touches the corpus, the package, the prompts or the golden set. Free, no credentials.

generation on every prompt change, with the tenant's gateway key. Make both required checks.

One variable, one row on the board, and the gate tells you which ones hurt

Variable	Flag
Chunk size 200	<code>build_index --max-tokens 200</code> , then evaluate
k = 2, 4 or 12	<code>--k 2</code> , <code>--k 4</code> , <code>--k 12</code>
Hybrid off, reranker off	<code>--no-hybrid</code> , <code>--no-rerank</code>
Minimum score 0.2	<code>--min-score 0.2</code>

Reranker off, measured

MRR 0.923 to 0.880; recall@1 loses 5 cases and gains none.

`REGRESSION GATE FAILED`, free, in seconds.

The store is a platform choice, not a quality one

NW_RETRIEVER	Searches	Track
<code>inprocess</code>	FAISS and BM25 over <code>artifacts/policy</code> , the default	any
<code>knowledge-base</code>	the tenant's Bedrock Knowledge Base on S3 Vectors	AWS
<code>rag-engine</code>	the tenant's RAG Engine corpus	Google Cloud
<code>ai-search</code>	the tenant's Azure AI Search index, hybrid	Azure
<code>qdrant</code>	the tenant's Qdrant collection on :6333	Local

The managed store returns candidates with scores from 0 to 1 on every track. The currency filter and the reranker still run in `retrieval.py`, on the metadata the upsert carried.

Every model call goes through one door that holds the key, the budget and the bill

```
curl -s $NW_GATEWAY_URL/key/info \
-H "Authorization: Bearer $NW_GATEWAY_KEY"
```

`spend`, `max_budget`, `key_alias`: your line on the platform. `GET /spend/keys` with the master key is every tenant's line.

The meter estimates, the gateway bills

`nw_policy_spend_usd_total` is the service's estimate from token counts and the price table. The gateway's `spend` is what the platform charges your tenant.

When the two disagree, explain the difference before you trust either.

The service closes the loop: a cache that cannot lie, feedback that becomes cases, drift that says when to look

Exact-match cache

Key: normalised question, audience, k, index hash, prompt version, model id. A rebuild or a prompt change empties it without a flush.

Not semantic: the Pro SLA is not the Enterprise SLA.

POST /feedback

`helpful`, `wrong` or `unsafe` on an `answer_id`, at most three per answer. `feedback --to-golden` prints the wrong ones in the golden set's shape; a person writes the answer.

/drift

Top-hit confidence PSI and the refusal ratio, from 200 requests. The first baseline is the golden set's, a placeholder until a week of traffic replaces it.

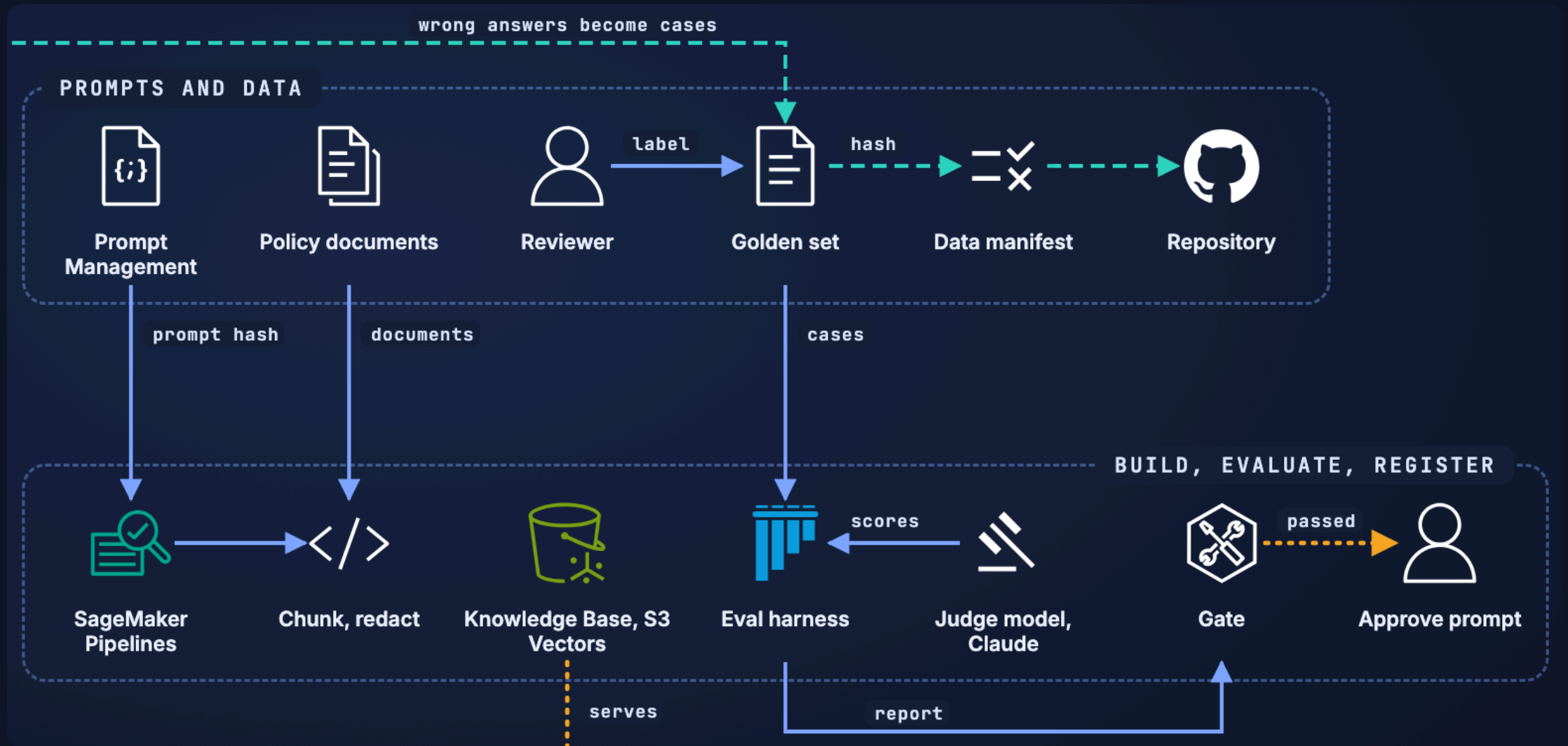
Six things get a version, and none of them is a weights file

Versioned	Where	Changes when
Prompt	<code>name@hash</code> in code; the same hash on the store's version, with its stage	one word of a prompt or rubric
Model id	by role in <code>nw/config.py</code> ; the gateway's model name	the track, a deprecation
Index manifest	corpus hash, chunker, embedder, reranker, prompt versions, drift baseline	the corpus, the chunker, either model
Vector store contents	the tenant's collection, filled from the index	the index
Golden set	111 cases, <code>golden_sha256_12</code> in every report	feedback becomes cases
Baseline	per mode and track, with the provenance the gate checks	a passed run you adopt

One contract per box, one implementation per track

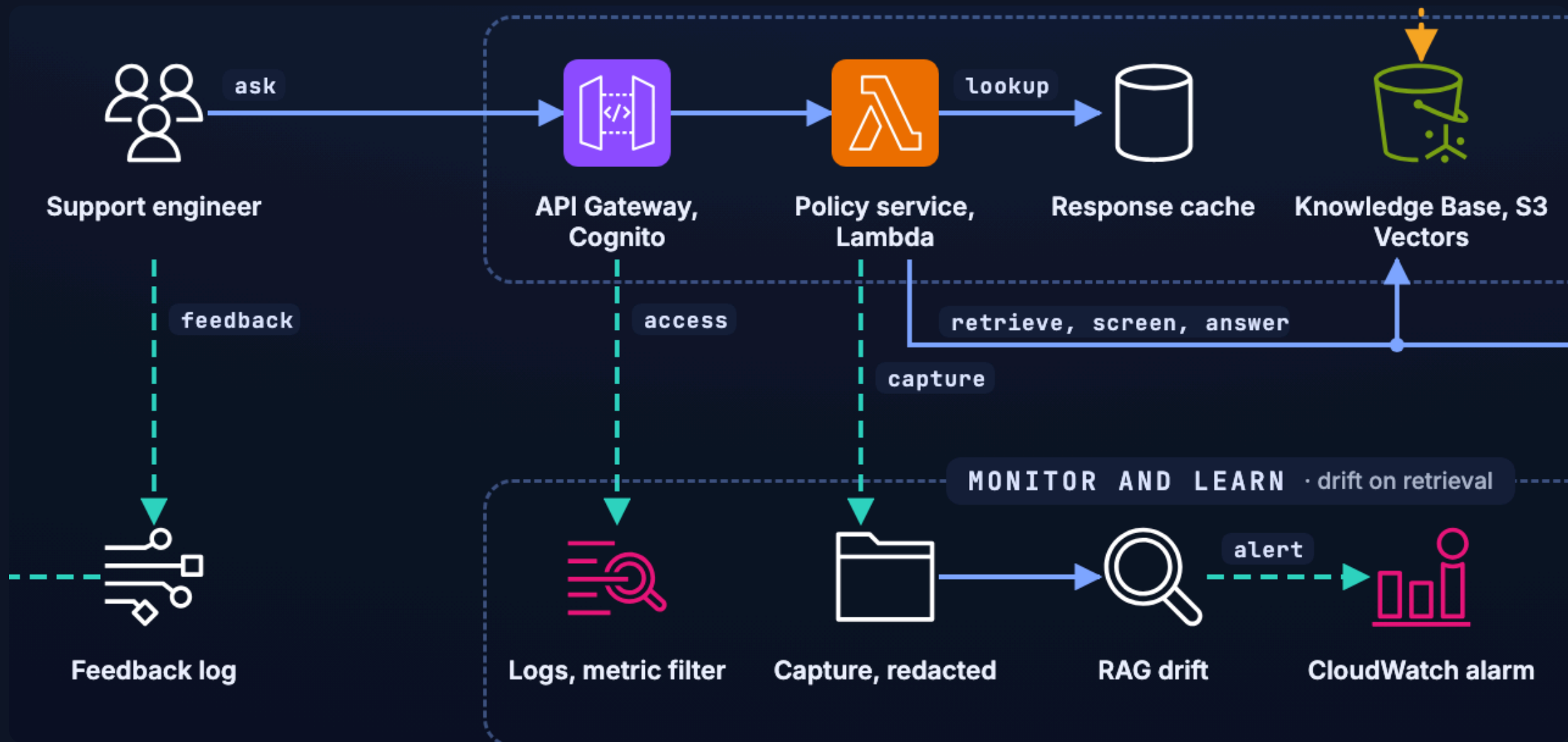
Contract	Local	AWS	Google Cloud	Azure
PromptStore	MLflow prompts	Bedrock Prompt Management	prompt management, stages in the bucket	blobs plus Azure ML data assets by hash
VectorStore	Qdrant	Knowledge Base on S3 Vectors	RAG Engine corpus	Azure AI Search index
Model gateway	LiteLLM	LiteLLM on Fargate behind CloudFront	LiteLLM on Cloud Run	API Management AI gateway
Workhorse, Judge	gpt-oss-20b; Claude or judge-free	gpt-oss-120b, Claude Opus 5	gpt-oss-120b, Claude Opus 5	gpt-oss-120b, Claude Opus 5 on Foundry
Screening on /ask	Llama Guard 3	Bedrock Guardrails	Model Armor	Prompt Shields
Drift	Evidently	alarm on drift_alert	log-based metric	log search alert on drift_alert

LLMOps, AWS: prompts, data, build and evaluate



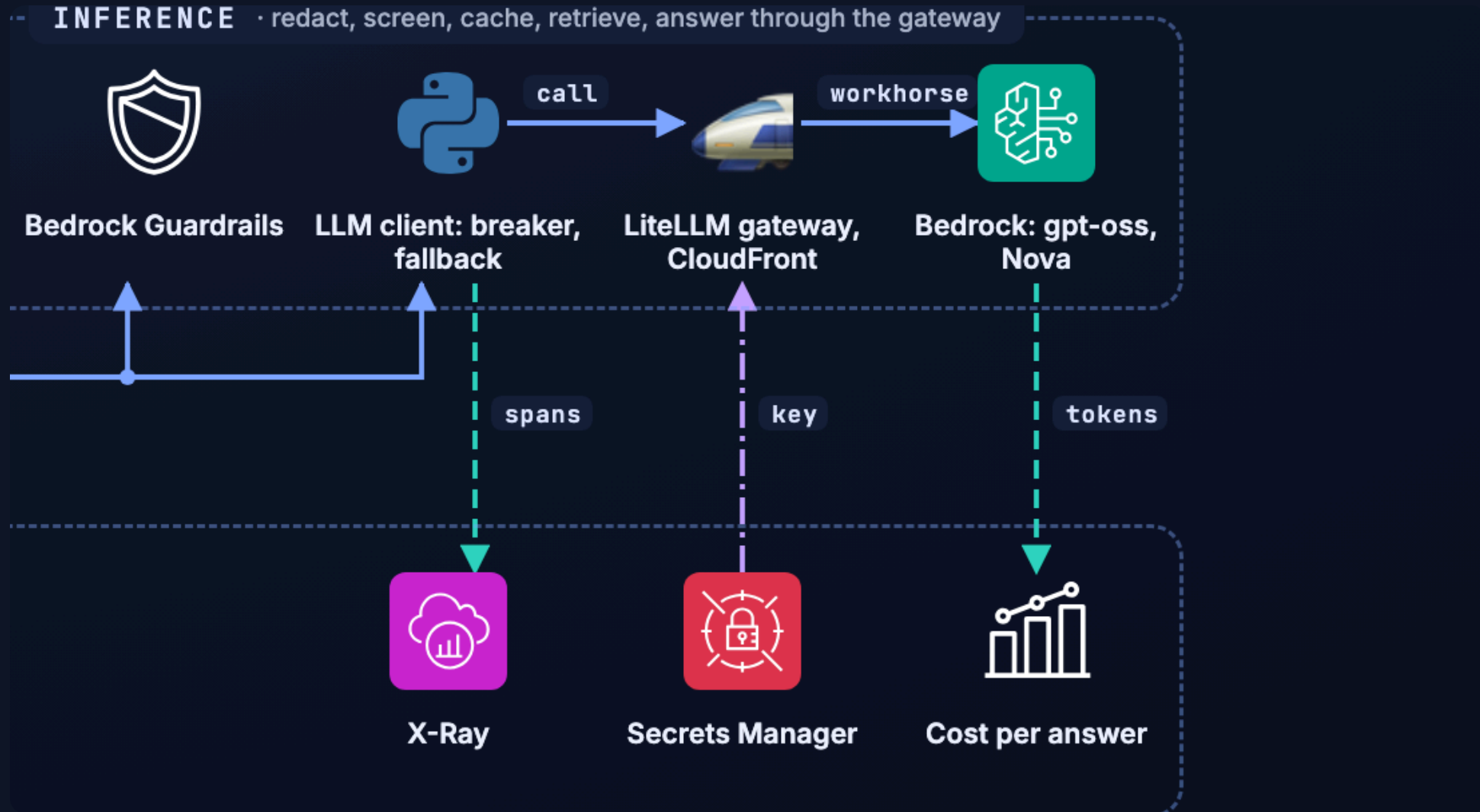
The top of the LLM Ops figure. A reviewer labels the golden set; the pipeline chunks and redacts into the Knowledge Base; the harness scores with the Judge; the gate passes a person the prompt to approve. Other tracks: the reserve appendix.

LLMOps, AWS: a question reaches the service and the store



API Gateway with Cognito in front of the policy Lambda, the response cache, the Knowledge Base; below, access logs, the redacted capture and RAG drift into a CloudWatch alarm; feedback into the log.

LLMOps, AWS: every model call through the guardrail and the gateway



Bedrock Guardrails screens the question; the LLM client holds the breaker and the fallback; LiteLLM behind CloudFront holds the key and the budget; Bedrock serves gpt-oss and Nova. Spans to X-Ray, the key from Secrets Manager, tokens into cost per answer.

Acceptance criteria

1. Sensitive fields never reach a chunk; dates do
2. The audience filter fails closed; citation validity is 1.0; every answer carries its prompt version
3. The store's prompt version matches `ANSWER_PROMPT.hash`, approved
4. The vector store's count matches the manifest
5. The retrieval gate passes, and fails with the reranker off

96 passed

what `make session04` prints when you are done

Questions worth asking

- You cut k from 8 to 2. Which metric moves first, and in which direction?
- The Judge scores 0.5 on an answer you think is correct. Who is right, and how would you find out?
- Faithfulness rises on Monday after someone sharpened the prompt on Friday. Which three things in the report tell you it was the prompt?
- The gate refuses your baseline because the Workhorse id changed. When is `--allow-model-change` the right answer?
- The store says one prompt version is approved and the manifest names another. Which one is serving?

RESERVE

Appendix

The rest of the landscape and the other tracks' figures. Use a slide when the room asks for it.

Reasoning models trade answer-time compute for accuracy

More thinking time

74 vs 12

percent on AIME 2024, one sample: o1 against GPT-4o (OpenAI, 2024)

Compute beats size, sometimes

With compute-optimal thinking at test time, a smaller model can outperform one **14 times larger** at matched compute, on problems it already partly solves.

Now open weights too

DeepSeek-R1 taught reasoning through reinforcement learning alone, then distilled it into small open models. Published in Nature.

For a support desk the trade is latency and tokens on every answer. Spend it on the hard ticket, not on the password reset.

Long context does not replace retrieval; it changes the price

	Long context	Retrieval
Quality, budget no object	higher on average	lower on average
Cost per question	every token, every time	a handful of passages
How it fails	the middle of the window gets lost	the right passage never arrives
Current and citable	whatever you pasted	filtered by metadata, cited by id

Route, do not pick

Letting the model choose retrieval or long context per query came close to long-context quality at much lower cost.

How fast windows grow is on the first part's landscape slide.

An LLM judge agrees with people about as often as people agree, with biases you can name

80%+

agreement of strong LLM judges with human preferences, the same rate at which humans agree with each other

- **Position:** prefers the answer shown first
- **Verbosity:** prefers the longer answer
- **Self-enhancement:** prefers answers from its own model family

The agreement rate was measured on someone else's task. Yours is measured on your labels: the calibration set of 102.

Prompt injection tops the OWASP list, and no fool-proof fix is known

OWASP LLM 2025	Where Project 3 answers it
1 Prompt Injection	the guardrail on <code>/ask</code> , context marked as data
2 Sensitive Information Disclosure	redaction, the audience from the key
3 Supply Chain	locked dependencies, pinned model revisions
4 Data and Model Poisoning	corpus hash in the manifest, <code>--check</code>
5 Improper Output Handling	structured output, citations validated

OWASP LLM 2025	Where Project 3 answers it
6 Excessive Agency	the next part: the approval gate
7 System Prompt Leakage	prompts hold no secrets; keys in the secret store
8 Vector and Embedding Weaknesses	the audience filter in the store, fails closed
9 Misinformation	refusal, key facts, the Judge, the gate
10 Unbounded Consumption	a budget per gateway key

Measured in support, the gain is real; measured on cost alone, it reverses

+14%

issues resolved per hour across 5,179 support agents with a generative AI assistant; +34 percent for novices, minimal for the most experienced

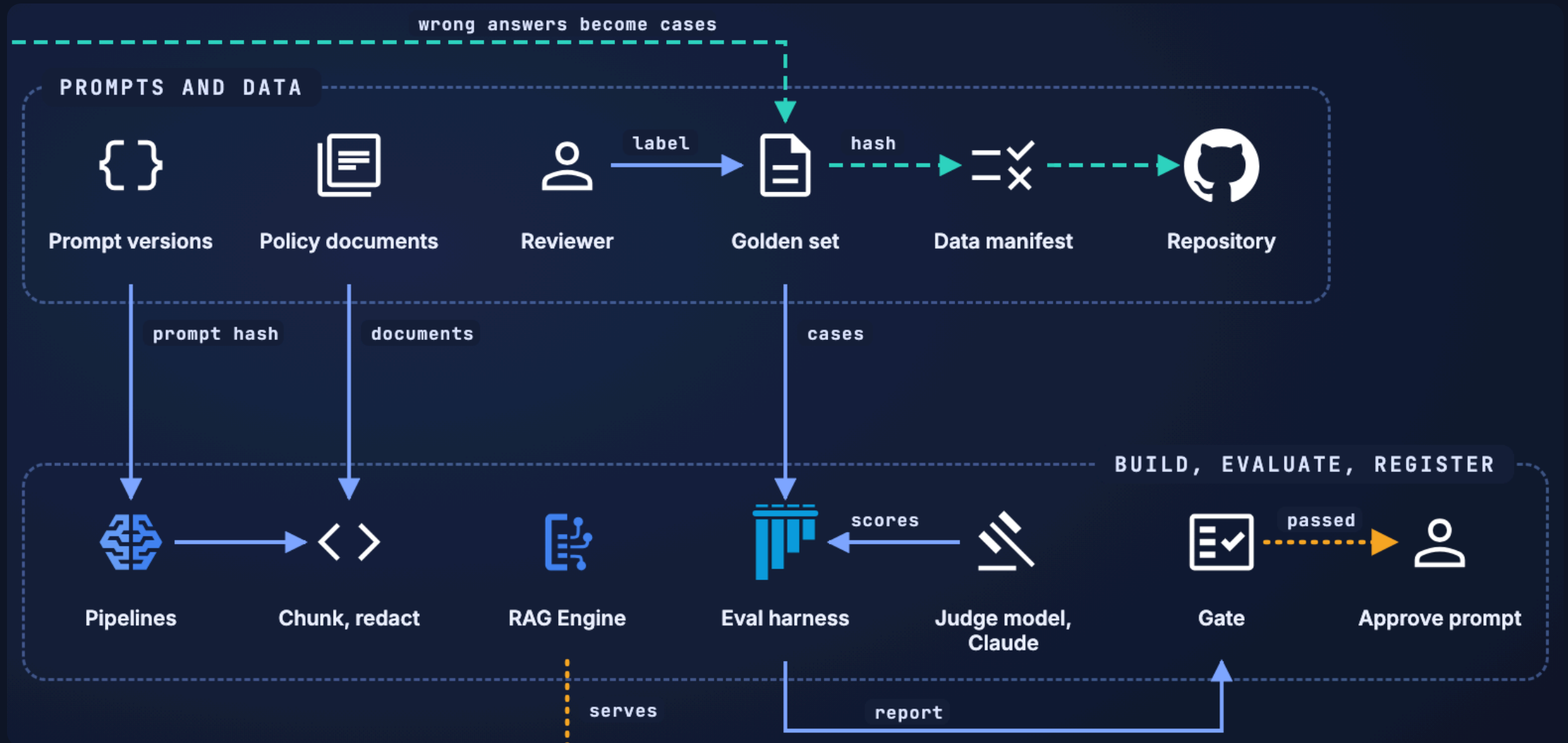
Brynjolfsson, Li and Raymond, The Quarterly Journal of Economics, 2025

The headline, then the correction

February 2024: Klarna said its assistant handled two thirds of its service chats in its first month, the work of 700 full-time agents.

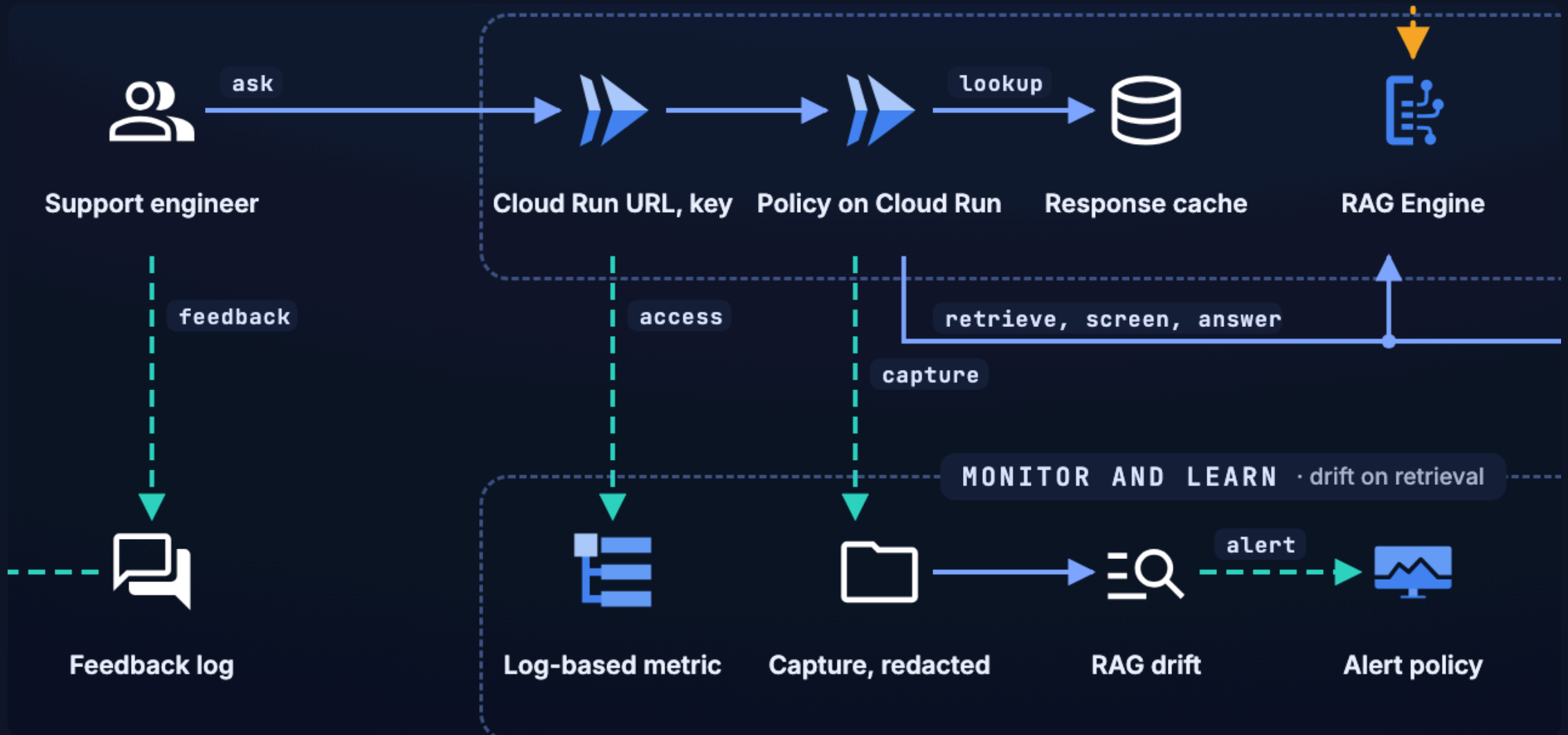
May 2025: its CEO was reported saying cost had been "a too predominant evaluation factor", leading to lower quality, and Klarna began hiring people again.

LLMOps, Google Cloud: prompts, data, build and evaluate



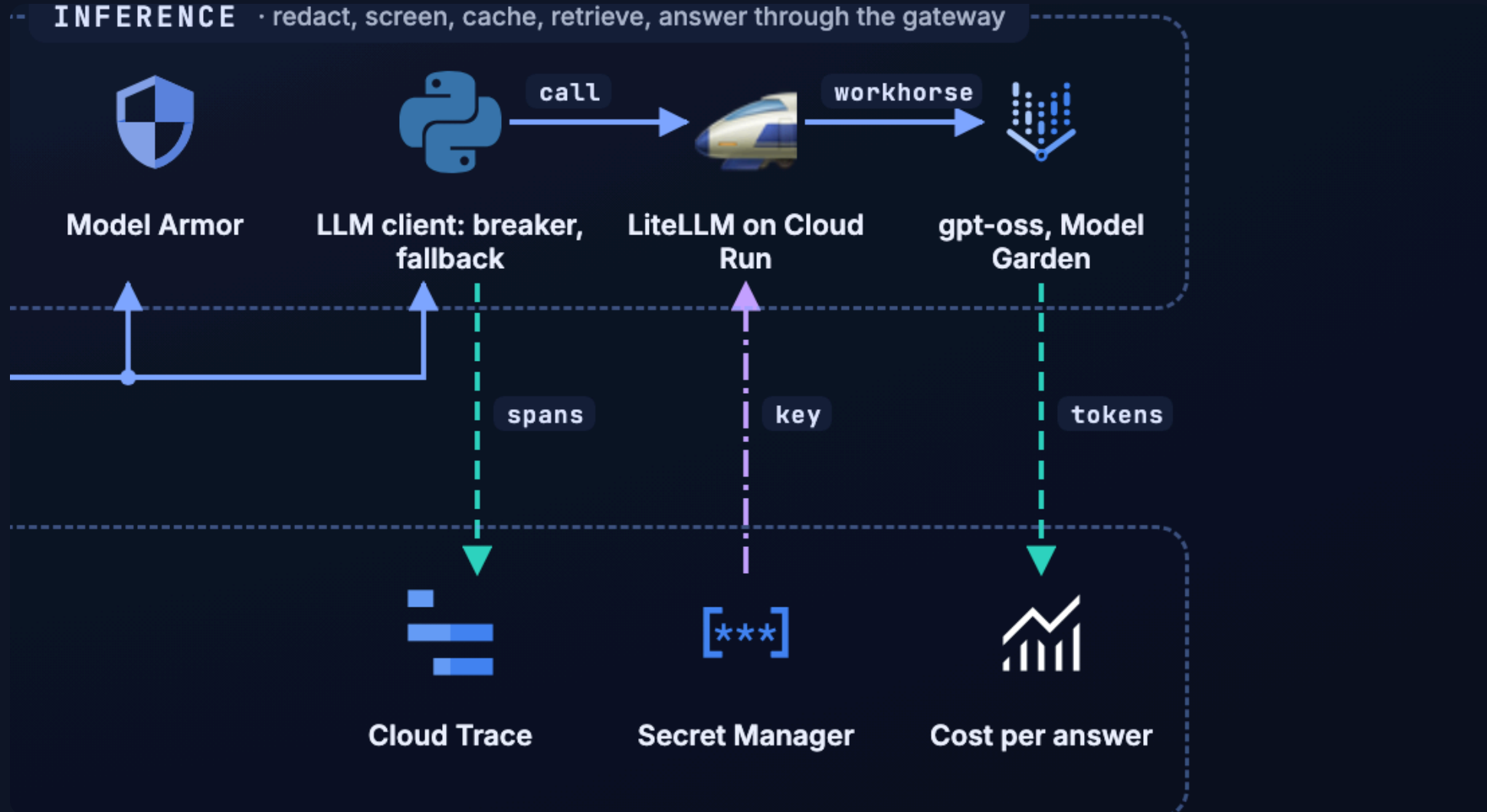
Prompt versions on the Agent Platform, Pipelines, the RAG Engine corpus, the harness with the Claude Judge and the gate.

LLMOps, Google Cloud: the question path



The Cloud Run URL with a key, the policy service on Cloud Run, the cache, the RAG Engine; log-based metrics, the redacted capture, RAG drift and an alert policy.

LLMOps, Google Cloud: the model path



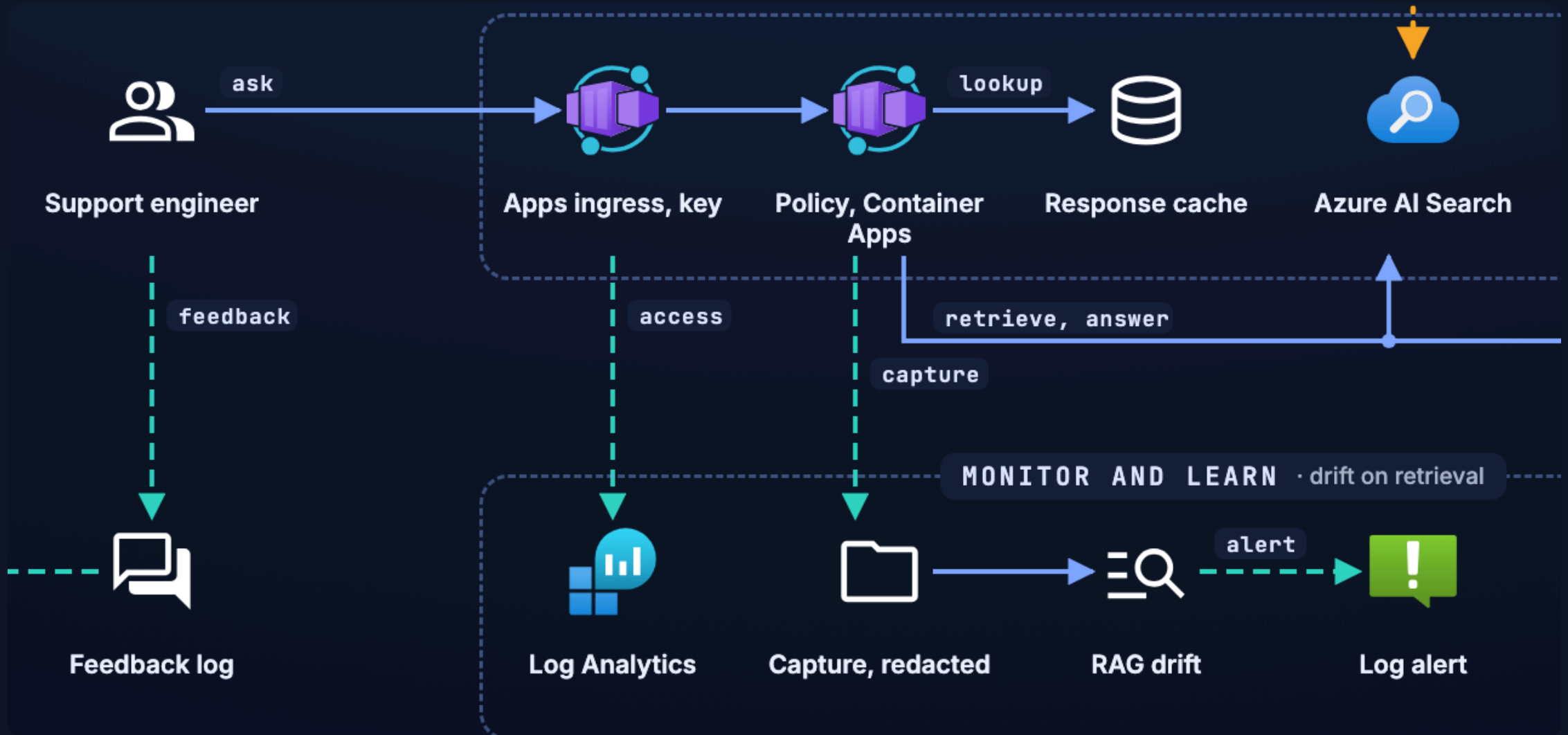
Model Armor screens the question; the client's breaker and fallback; LiteLLM on Cloud Run; gpt-oss from Model Garden. Cloud Trace, Secret Manager, cost per answer.

LLMOps, Azure: prompts, data, build and evaluate



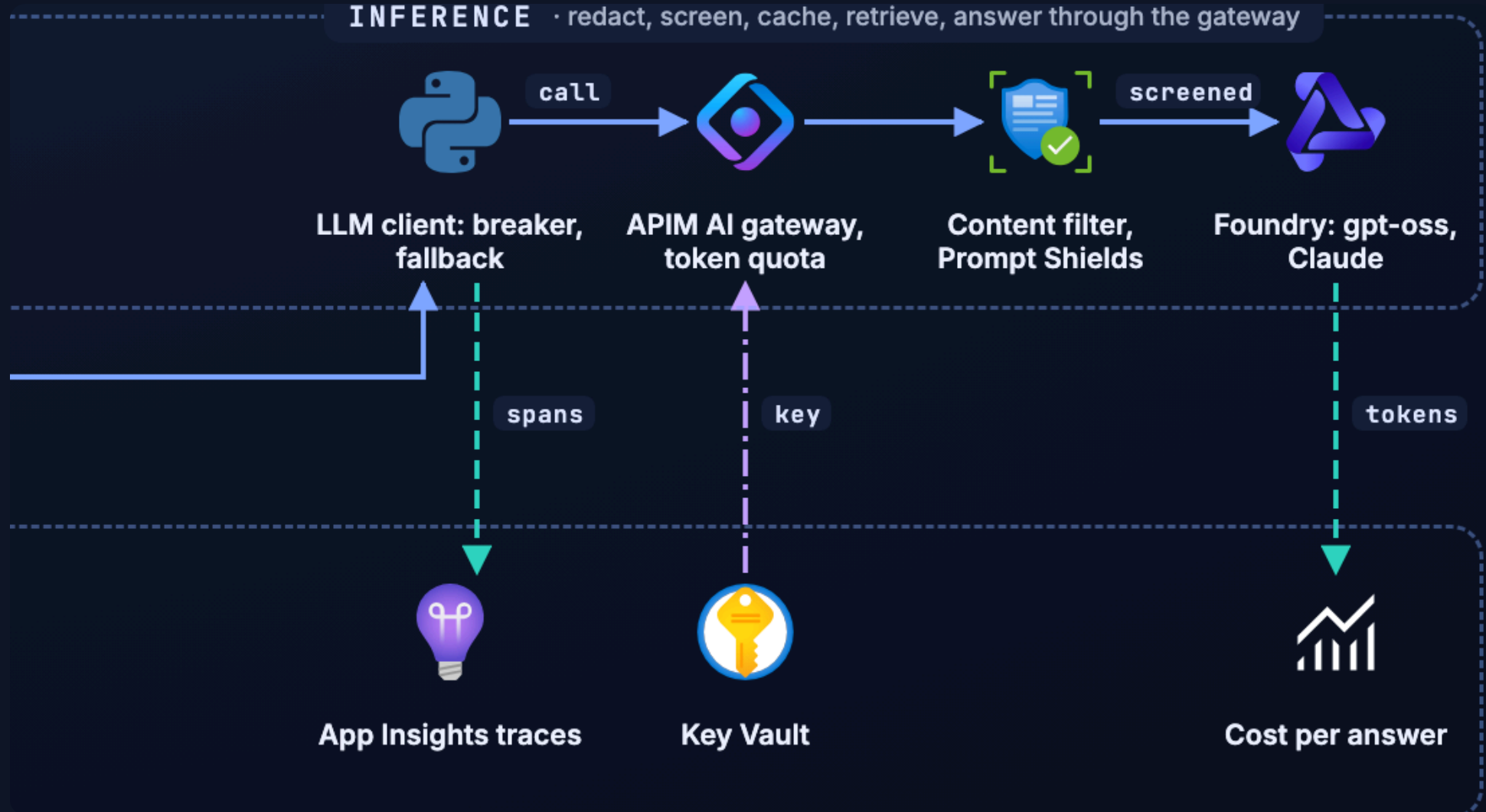
Prompt versions as blobs, Azure Machine Learning pipelines, the Azure AI Search index, the harness, the Judge and the gate.

LLMOps, Azure: the question path



Container Apps ingress with a key, the policy service on Container Apps, the cache, Azure AI Search; Log Analytics, the capture, RAG drift and a log alert.

LLMOps, Azure: the model path



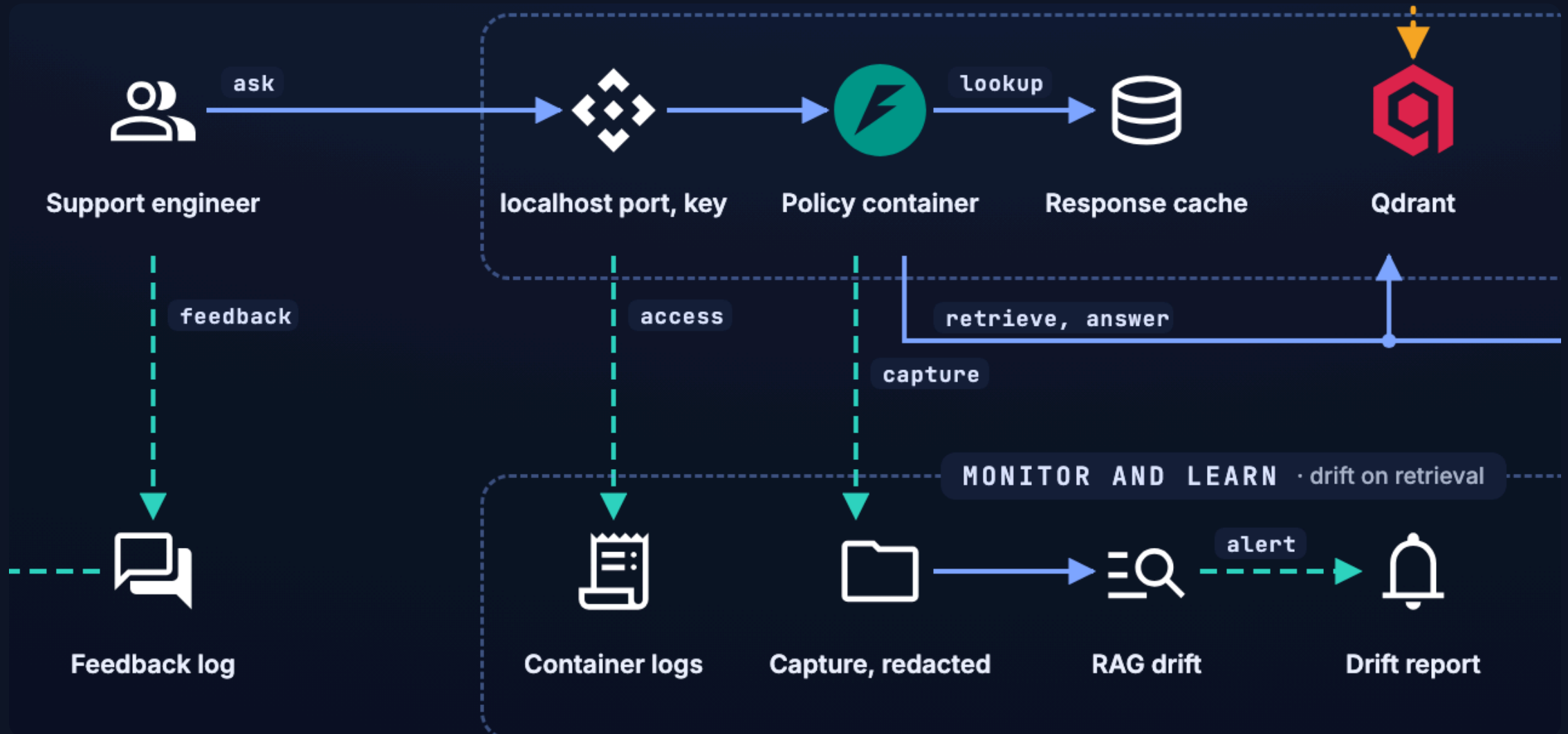
API Management's AI gateway with a token quota, the content filter with Prompt Shields, Foundry serving gpt-oss and Claude; Application Insights traces, Key Vault, cost per answer.

LLMOps, Local: prompts, data, build and evaluate



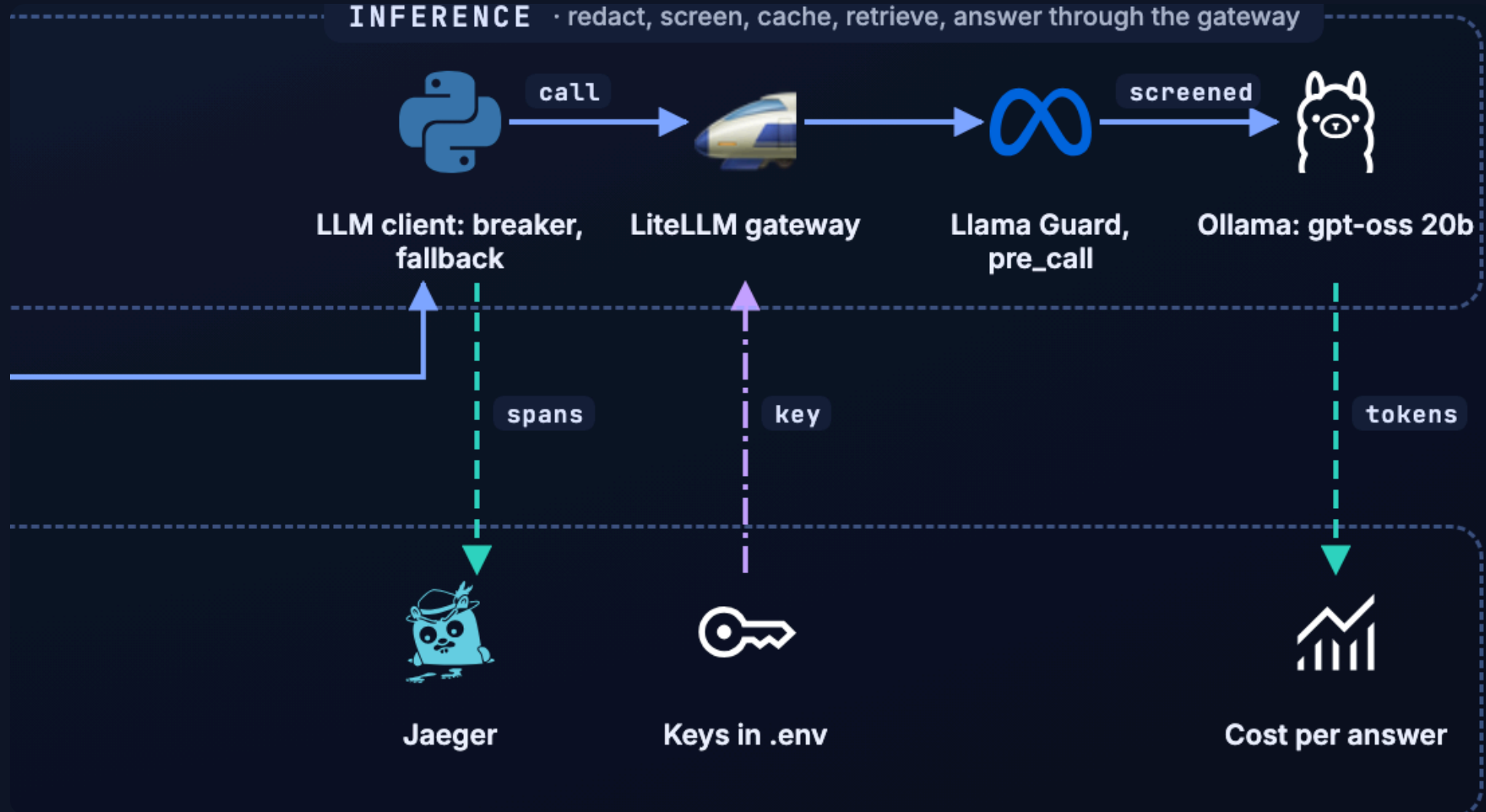
MLflow prompt versions, the Kubeflow local runner, Qdrant, the harness and the gate; the Judge only with a cloud key.

LLMOps, Local: the question path



A localhost port with a key, the policy container, the cache, Qdrant; container logs, the capture, RAG drift and the drift report.

LLMOps, Local: the model path



The client's breaker and fallback, LiteLLM, Llama Guard as a pre-call check, gpt-oss-20b on Ollama; Jaeger, keys in .env, cost per answer.